PYSPARK COMPLETE NOTES

PHASE 3

Intermediate Transformations

Combine DataFrames, work with strings, dates, nulls and array columns.

YOU WILL

- ✓ Stack DataFrames with union and unionByName - and know the difference
- ✓ Use string and date functions, including date_format
- ✓ Drop or fill nulls, for all columns or a subset
- ✓ Split text into arrays, index them and explode them into rows
- ✓ Find hidden missing values such as \N and turn them into real nulls

START HERE IF

Phase 2 (withColumn, select, filter).

Built from a Databricks PySpark tutorial notebook and its course outline.
Clickable contents, links and bookmarks work in any PDF viewer.

Phase 3 · Intermediate Transformations

Combine DataFrames, work with strings, dates, nulls and array columns.

YOU WILL BE ABLE TO

- ✓ Stack DataFrames with union and unionByName - and know the difference
- ✓ Use string and date functions, including date_format
- ✓ Drop or fill nulls, for all columns or a subset
- ✓ Split text into arrays, index them and explode them into rows
- ✓ Find hidden missing values such as \N and turn them into real nulls

START HERE IF

Phase 2 (withColumn, select, filter).

IN THIS PDF

Explanations, diagrams and code from the notebook · sections tagged **Beyond the notebook** · a decoder for every term · interview questions · a data-engineering lens · a self-test with answers · a glossary

How to read the boxes

IN PLAIN WORDS A jargon-free explanation of the idea.	DECODE THE CODE Meaning of every function, option and symbol in the code above (first time it appears).
TIP A habit that makes your code cleaner, faster or safer.	WATCH OUT A trap that gives a wrong or surprising result.
NOTEBOOK OBSERVATION Something specific noticed in the course notebook.	DATA ENGINEERING LENS How the skill is used in real pipelines (summary page at the end).
EXTRA Extra knowledge that is not in the notebook.	BEYOND THE NOTEBOOK A tag under a heading: new material added to the notebook's topics, ending with an Interview corner.

CONTENTS (click an entry to jump to it)

3.1 union() and unionByName() . . . 3
3.2 String functions . . . 5
3.3 Date functions . . . 6
3.4 Handling nulls . . . 8
3.5 split() and indexing . . . 9
3.6 explode() and array_contains() . . . 10
3.7 Null values vs missing-value markers . . . 11
From notebook to pipeline . . . 15
Glossary . . . 15
Check yourself . . . 18
Answers and solutions . . . 19

3.1 union() and unionByName()

Union stacks the rows of one DataFrame underneath another - same columns, more rows. The notebook first builds two tiny DataFrames to experiment on.

Preparing the DataFrames

PYSPARK

```
data1 = [('1','kad'),
         ('2','sid')]
schema1 = 'id STRING, name STRING'

df1 = spark.createDataFrame(data1,schema1)

data2 = [('3','rahul'),
         ('4','jas')]
schema2 = 'id STRING, name STRING'

df2 = spark.createDataFrame(data2,schema2)
```

DECODE THE CODE

`[('1','kad'), ...]`	A Python list of tuples. Each tuple is one row of the DataFrame.
`STRING / DOUBLE / INT`	SQL-style type names: text, decimal number, whole number.
`spark`	The SparkSession - your entry point to Spark. Databricks creates it for you, already named **spark**.
`spark.createDataFrame(rows, schema)`	Builds a DataFrame from Python data.
`spark.createDataFrame(data, schema)`	Builds a small DataFrame from a Python list plus a schema string.

`spark.createDataFrame(data, schema)` builds a DataFrame from a Python list of tuples plus a DDL schema string - the same DDL idea from Phase 1.

df1

id	name
1	kad
2	sid

df2

id	name
3	rahul
4	jas

union() - works by position

PYSPARK

```
df1.union(df2).display()
```

DECODE THE CODE

`.union(other)`	Stacks the rows of two DataFrames, matching columns by **position**.
`.display()`	Databricks' table viewer. It is an **action**, so it makes Spark run the job.

SAMPLE OUTPUT

id	name
1	kad
2	sid
3	rahul
4	jas

Both DataFrames have the columns in the same order, so the result is exactly what you expect. Now the notebook **re-creates df1 with the columns swapped**:

PYSPARK

```
data1 = [('kad','1',),
         ('sid','2',)]
schema1 = 'name STRING, id STRING'

df1 = spark.createDataFrame(data1,schema1)

df1.display()
```

PYSPARK

```
df1.union(df2).display()
```

RESULT OF union() - VALUES LAND IN THE WRONG COLUMNS

name	id
kad	1
sid	2
3	rahul
4	jas

`union` does not look at column names. It simply glues column 1 to column 1 and column 2 to column 2. Since df1 is now (name, id) and df2 is still (id, name), the ids **3** and **4** end up in the name column and the names end up in id. Both columns are strings here so Spark does not complain - you just get silently wrong data.

unionByName() - works by column name

PYSPARK

```
df1.unionByName(df2).display()
```

DECODE THE CODE

`.unionByName(other)`	Stacks the rows of two DataFrames, matching columns by **name**.

RESULT OF unionByName() - CORRECT

name	id
kad	1
sid	2
rahul	3
jas	4

	union()	unionByName()
Matches columns by	**Position**	**Name**
Column order must match	Yes	No
Removes duplicates?	No (behaves like SQL UNION ALL)	No
Use when	You are sure the layouts are identical	Column order may differ - the safer default

COLUMNS THAT EXIST IN ONLY ONE DATAFRAME

PYSPARK · EXTRA

```
df1.unionByName(other_df, allowMissingColumns=True)   # missing columns are filled with null
```

DF1 AND DF2 ARE REUSED

The same names `df1` and `df2` are overwritten again for the joins in Phase 4. If you re-run an earlier cell later, the DataFrame may not hold what you think.

union() - by POSITION

df1

name	id
kad	1
sid	2

df2

id	name
3	rahul
4	jas

same colour = columns that get stacked together
matched by POSITION (1st with 1st, 2nd with 2nd)

result

name	id
kad	1
sid	2
3	rahul
4	jas

ids 3 and 4 landed in the NAME column and the names in the ID column. No error - silently wrong.

`df1.union(df2)`

unionByName() - by NAME

df1

name	id
kad	1
sid	2

df2

id	name
3	rahul
4	jas

same colour = columns that get stacked together
matched by NAME

result

name	id
kad	1
sid	2
rahul	3
jas	4

Correct: each value is in its own column, even though df1 and df2 list the columns in a different order.

`df1.unionByName(df2)`

3.2 String functions

PYSPARK

```
df.select(upper('Item_Type').alias('upper_Item_Type')).display()
```

DECODE THE CODE

`.select(...)`	Keeps only the listed columns (or new calculated columns) - like SQL SELECT.
`upper(col)`	Converts text to CAPITALS.
`.alias('new')`	Gives a column a new name **in this result only**.

`upper(column)` converts text to capitals. Chained with `.alias` it names the output column. Spark has a family of such functions:

Function	Does	'soft drinks' becomes
`upper('c')`	ALL CAPITALS (used in the notebook)	SOFT DRINKS
`lower('c')`	all lower case	soft drinks
`initcap('c')`	Capital first letter of every word	Soft Drinks

NOTEBOOK OBSERVATION

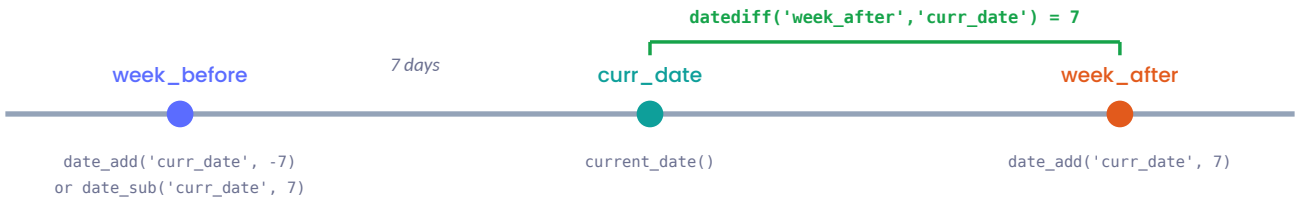
The section is headed **Initcap()**, but the cell actually calls `upper()`. They are different: `initcap` would give "Soft Drinks" whereas `upper` gives "SOFT DRINKS". The correct initcap version is `df.select(initcap('Item_Type'))`.

NO COL() NEEDED

Spark functions accept either a column name as a string (`upper('Item_Type')`) or a column object (`upper(col('Item_Type'))`).

3.3 Date functions

The notebook builds a small family of date columns on top of `current_date()`. Each step is assigned back to `df`, so the columns accumulate.



datediff(end, start): the LATER date goes first.

current_date()

PYSPARK

```
df = df.withColumn('curr_date',current_date())

df.display()
```

DECODE THE CODE

`.withColumn(name, expression)`	**Adds** a column, or **replaces** the one with the same name.
`current_date()`	Today's date (a date, no time).

Adds today's date (a **date** type, no time) to every row.

date_add() - move forward

PYSPARK

```
df = df.withColumn('week_after',date_add('curr_date',7))

df.display()
```

DECODE THE CODE

`date_add(col, n)`	Adds n days to a date. A **negative** n goes back.

date_sub() - move backward

PYSPARK

```
df.withColumn('week_before',date_sub('curr_date',7)).display()
```

DECODE THE CODE

`date_sub(col, n)`	Subtracts n days from a date.

PYSPARK

```
df = df.withColumn('week_before',date_add('curr_date',-7))

df.display()
```

`date_sub('curr_date', 7)` and `date_add('curr_date', -7)` give the same answer: 7 days earlier. The first version is only displayed; the second is assigned, so **that** is the one stored in `week_before`.

datediff() - days between two dates

PYSPARK

```
df = df.withColumn('datediff',datediff('week_after','curr_date'))

df.display()
```

DECODE THE CODE

`datediff(end, start)`	Days between two dates: **end minus start**.

`datediff(end, start)` returns end minus start in days. The **later** date goes first. week_after is 7 days after curr_date, so every row shows **7**.

date_format() - show a date as text

PYSPARK

```
df = df.withColumn('week_before',date_format('week_before','dd-MM-yyyy'))

df.display()
```

DECODE THE CODE

`date_format(col, 'dd-MM-yyyy')`	Shows a date as **text** using a pattern. **MM** = month, **mm** = minutes.

ILLUSTRATIVE ROW IF TODAY WERE 2026-10-15 (AFTER date_format)

curr_date	week_after	week_before	datediff
2026-10-15	2026-10-22	08-10-2026	7

Pattern letters	Meaning	Example
`yyyy`	4-digit year	2026
`MM`	**Month** as two digits (capital M)	10
`dd`	Day of month	08
`mm`	**Minutes** (lower-case m)	05

DATE_FORMAT RETURNS TEXT, AND M IS NOT M

After `date_format`, `week_before` is a **string** like "08-10-2026" - you can no longer do date arithmetic on it. And `MM` is months while `mm` is minutes; mixing them up is a classic bug.

3.4 Handling nulls

A **null** means "no value". Real data is full of them (in this dataset, Outlet_Size is a column that has nulls, which is why the notebook uses it in its examples). You can either **drop** rows or **fill** the gaps.

Dropping nulls - dropna()

PYSPARK

```
df.dropna('all').display()
```

DECODE THE CODE

`.dropna(how / subset)`	Drops rows containing nulls: **'any'** null, **'all'** nulls, or only in the *subset* columns.

PYSPARK

```
df.dropna('any').display()
```

PYSPARK

```
df.dropna(subset=['Outlet_Size']).display()
```

DECODE THE CODE

`subset=[...]`	Restricts the operation to the listed columns only.

PYSPARK

```
df.display()
```

Call	Drops a row when ...
`dropna('all')`	**every** column in the row is null (rarely happens).
`dropna('any')`	**at least one** column is null (this is also the default).
`dropna(subset=['Outlet_Size'])`	Outlet_Size is null; nulls in other columns are ignored.

The final `df.display()` confirms none of these changed `df` - nothing was assigned.

Filling nulls - fillna()

PYSPARK

```
df.fillna('NotAvailable').display()
```

DECODE THE CODE

`.fillna(value, subset)`	Replaces nulls with a value (a text value only fills string columns).

PYSPARK

```
df.fillna('NotAvailable',subset=['Outlet_Size']).display()
```

Call	Effect
`fillna('NotAvailable')`	Replaces nulls with the text in **every string column**. Number and date columns are untouched because a text value does not fit them.
`fillna('NotAvailable', subset=['Outlet_Size'])`	Only fills the listed column(s).

UNRESTRICTED FILLNA TOUCHES MORE THAN YOU THINK

Item_Weight is a string column in this notebook, so `fillna('NotAvailable')` without a subset also puts the text "NotAvailable" into the null weights. Using `subset` keeps the change to the column you mean.

FILLING NUMBERS

PYSPARK · EXTRA

```
df.fillna(0, subset=['Item_Visibility'])      # a number fills numeric columns
```

DF WITH NULLS

Item_Id	Outlet_Size	Location
FDA15	Medium	Tier 1
DRC01	null	Tier 2
FDN15	Medium	null
FDX07	null	null

FILLNA(..., SUBSET=['OUTLET_SIZE'])

Item_Id	Outlet_Size	Location
FDA15	Medium	Tier 1
DRC01	NotAvailable	Tier 2
FDN15	Medium	null
FDX07	NotAvailable	null

DROPNA('ANY') -> ANY NULL DROPS THE ROW

Item_Id	Outlet_Size	Location
FDA15	Medium	Tier 1

DROPNA(SUBSET=['OUTLET_SIZE'])

Item_Id	Outlet_Size	Location
FDA15	Medium	Tier 1
FDN15	Medium	null

only Outlet_Size is checked

DROPNA('ALL') -> ONLY ROWS THAT ARE ALL NULL

no row here is all-null, so nothing is dropped

dropna removes ROWS. fillna keeps every row and replaces the gaps (only in the columns you allow).

3.5 split() and indexing

`split(column, pattern)` cuts a string into pieces and returns an **array** column. The pattern is a regular expression; here it is a single space.

PYSPARK

```
df.withColumn('Outlet_Type',split('Outlet_Type',' ')).display()
```

DECODE THE CODE

`split(col, pattern)`	Cuts a string into pieces and returns an **array** column.

Indexing - pick one element

PYSPARK

```
df.withColumn('Outlet_Type',split('Outlet_Type',' ')[1]).display()
```

DECODE THE CODE

`...[1]`	Array index. Counting starts at **0**, so [1] is the second element.

WHAT HAPPENS TO TWO OUTLET TYPE VALUES

Outlet_Type (original)	after split	after split(...)[1]
Supermarket Type1	[Supermarket, Type1]	Type1
Grocery Store	[Grocery, Store]	Store

- Array positions start at **0**: `[0]` is the first word, `[1]` the second.

- The notebook writes the result back into the `Outlet_Type` column name with `withColumn`, replacing the original in that output. As it is not assigned to `df`, the real column is untouched.

3.6 explode() and array_contains()

`explode` takes an array column and turns **each element into its own row**, repeating the other columns. It is the opposite of `collect_list` (Phase 4).

1 A TEXT COLUMN

Item_Id	Outlet_Type
FDA15	Supermarket Type1

split('Outlet_Type', ' ')

2 AN ARRAY COLUMN (ONE CELL, TWO ITEMS)

Item_Id	Outlet_Type
FDA15	[Supermarket, Type1]

explode('Outlet_Type')

3 ONE ROW PER ITEM

Item_Id	Outlet_Type
FDA15	Supermarket
FDA15	Type1

Index: split(...)[1]

```
[Supermarket, Type1]   index 0 = Supermarket
                       index 1 = Type1
```

counting starts at 0

array_contains('Outlet_Type', 'Type1')

```
[Supermarket, Type1]  ->  true
[Supermarket, Type2]  ->  false
```

a True/False flag you can filter on

explode changes the number of rows!

PYSPARK

```
df_exp = df.withColumn('Outlet_Type',split('Outlet_Type',' '))

df_exp.display()
```

PYSPARK

```
df_exp.withColumn('Outlet_Type',explode('Outlet_Type')).display()
```

DECODE THE CODE

`explode(col)`	Turns each element of an array into its **own row**.

BEFORE EXPLODE: ONE ROW

Item_Identifier	Outlet_Type (array)
FDA15	[Supermarket, Type1]

AFTER EXPLODE: TWO ROWS

Item_Identifier	Outlet_Type
FDA15	Supermarket
FDA15	Type1

PYSPARK

```
df_exp.display()
```

Displaying `df_exp` again shows it still holds the arrays: the exploded version was never assigned.

array_contains() - does the array hold a value?

PYSPARK

```
df_exp.withColumn('Type1_flag',array_contains('Outlet_Type','Type1')).display()
```

DECODE THE CODE

`array_contains(col, value)`	True / False: does the array hold this value?

ILLUSTRATIVE RESULT

Outlet_Type	Type1_flag
[Supermarket, Type1]	true
[Supermarket, Type2]	false
[Grocery, Store]	false

`array_contains(array_column, value)` returns **true** or **false** per row - a handy flag you can filter on.

> **EXPLODE CHANGES THE NUMBER OF ROWS**
> Exploding two-element arrays doubles the row count. Aggregating afterwards (for example summing sales) would count the same item twice, so explode only when you really want one row per element.

3.7 Null values vs missing-value markers

BEYOND THE NOTEBOOK

Section 3.4 handled **real nulls**. Real files also hide missing values behind **markers**: text that *means* 'unknown' but that Spark reads as ordinary data. Common ones are `\N` (database exports), `NA` and `N/A` (spreadsheets), the word `null`, `-` and blank text.

RAW CSV TEXT

```
Item_Id,Weight,Size
FDA15,9.3,Medium
DRC01,\N,Small
FDN15,NA,
FDX07,null,High
```

DEFAULT READ

Item_Id	Weight	Size
FDA15	9.3	Medium
DRC01	\N	Small
FDN15	NA	NULL
FDX07	null	High

only the EMPTY field became a real null
'\N', 'NA', 'null' are just strings

clean →

AFTER CLEANING MARKERS

Item_Id	Weight	Size
FDA15	9.3	Medium
DRC01	NULL	Small
FDN15	NULL	NULL
FDX07	NULL	High

now isNull(), dropna(), fillna()
and a numeric cast all behave

WHY THE MARKERS MATTER

- **Filters** — isNull() misses them, so rows slip through
- **Casts** — cast('double') errors in Spark 4 (null in Spark 3)
- **Counts** — count(Weight) counts '\N' as a real value
- **Joins / keys** — 'NA' = 'NA' matches - unrelated rows join

Where a marker comes from: database exports (\N), spreadsheets (NA, N/A), systems that write the word null, or '-' and '?'.

Value in the CSV	Spark reads it as	isNull() sees it?
an empty field (,,)	a real **null**	Yes
`\N`	the text **\N**	**No**
`NA` or `N/A`	the text	**No**
`null` as a word	the text **null**	**No**
spaces only	the text of spaces	**No**

These results come from reading a small file with `header=True` (Spark 4.0.1):

DEFAULT READ · ONLY THE EMPTY FIELD IS A REAL NULL

Item_Id	Weight	Size
FDA15	9.3	Medium
DRC01	\N	Small
FDN15	NA	NULL
FDX07	null	High

MARKERS PASS EVERY NULL CHECK

Filters: `isNull()` and `dropna()` miss them, so rows slip through.

Casts: `cast('double')` on `\N` raises *CAST_INVALID_INPUT* in Spark 4 (ANSI mode is on by default) and gives **null** in older Spark 3 setups. Either way the marker was never handled on purpose.

Counts and averages: `count(col)` counts a marker as a real value.

Joins: two rows both holding `NA` match each other.

Option 1: tell the reader which text means null

PYSPARK · nullValue TURNS ONE MARKER INTO A REAL NULL AT READ TIME

```
df = spark.read.format('csv')\
          .option('header', True)\
          .option('nullValue', '\\N')\
          .load('/FileStore/tables/BigMart_Sales.csv')
```

DECODE THE CODE

`spark.read`	Starts a **DataFrameReader**: the object you configure and then use to load a file.
`.format('csv')`	Chooses the file format: **csv**, **json**, **parquet** ... (the same call is used when writing).
`\ (backslash)`	Line continuation: the statement carries on in the next line. Nothing may follow it on the same line.
`.option(key, value)`	Sets **one** setting for the reader or writer. Call it again for each extra setting.
`header`	**True** = the first row of the CSV holds column names, not data.
`nullValue`	CSV option: the text to read as a real null (for example \N).
`.load(path)`	Points Spark at the file or folder and returns a DataFrame. (Lazy until an action needs the data.)
`/FileStore/tables/`	The Databricks folder (DBFS) where uploaded files live.

`nullValue` accepts **one** text. In a test on Spark 4.0.1, empty fields still read as null after setting it, but check this on your own file. For several markers, clean after reading.

Option 2: clean many markers after the read

PYSPARK · A REUSABLE MARKER CLEANER

```
markers = ['\\N', 'NA', 'N/A', 'null', 'NULL', '-', '']

def markers_to_null(df, cols, markers=markers):
    for c in cols:
        df = df.withColumn(c, when(trim(col(c)).isin(markers), None).otherwise(col(c)))
    return df

df_clean = markers_to_null(df, ['Item_Weight', 'Outlet_Size'])
```

DECODE THE CODE

`def name(args):`	Defines an ordinary Python function.
`when(condition, value)`	An if / else-if branch for a new column - like SQL CASE WHEN. First match wins.
`trim(col)`	Removes spaces from both ends of a string.
`col('name')`	Refers to a column by name so you can do something with it (rename, compare, calculate).
`.isin(a, b)`	True when the value is in the given list.
`.otherwise(value)`	The **else**: used for every row that matched no earlier when().

AFTER CLEANING · FDP10 HAD THE MARKER \N

Item_Identifier	Item_Weight	Outlet_Size
FDA15	9.3	Medium
FDP10	NULL	Medium
FDX07	19.2	NULL

HOW THE CLEANER READS

`trim(col(c))` removes spaces, `.isin(markers)` tests membership in the list, `when(..., None)` writes a real null, and `.otherwise(col(c))` keeps every other value unchanged.

Do the cast **after** the cleaning: `col('Item_Weight').cast('double')`.

Option 3: measure before you clean

PYSPARK · COUNT REAL NULLS AND MARKERS PER COLUMN

```
def profile_nulls(df, cols, markers=markers):
    exprs = []
    for c in cols:
        exprs.append(sum(when(col(c).isNull(), 1).otherwise(0)).alias(c + '_nulls'))
        exprs.append(sum(when(trim(col(c)).isin(markers), 1).otherwise(0)).alias(c + '_markers'))
    return df.agg(*exprs)

profile_nulls(df, ['Item_Weight', 'Outlet_Size']).display()
```

DECODE THE CODE

`sum(col)`	Total of a column (Spark's own *sum*, available through the import *).
`.isNull()`	True where the value is missing. (A null is never *equal* to anything, so == *None* fails.)
`.agg(...)`	Calculates one or more aggregates for every group.

OUTPUT · RAW READ OF A 10-ROW SAMPLE (Spark 4.0.1)

Item_Weight_nulls	Item_Weight_markers	Outlet_Size_nulls	Outlet_Size_markers
0	1	3	0

The profile shows **0 nulls but 1 marker** in Item_Weight: invisible to `isNull()`, obvious here. After `markers_to_null` the same profile reports 1 null and 0 markers.

WHERE THIS BELONGS IN A PIPELINE

Convert markers to nulls once, at the boundary where raw data becomes clean data (the **Bronze to Silver** step in Phase 7). Keep the marker list in one place and write it down - it is part of the data contract with the source system.

CAST WITHOUT CRASHING

To get null instead of an error for unparseable text, use SQL's `try_cast`: `expr('try_cast(Item_Weight as double)')`. A plain cast raises an error when ANSI mode is on (the default in Spark 4).

INTERVIEW CORNER

Q What is the difference between null, an empty string and a marker like \N?

A A null is a true missing value Spark understands. An empty string is a real (empty) text. A marker is text that *means* missing to humans but that Spark treats as data.

Q How would you find hidden missing values in a new dataset?

A Profile each column: count real nulls, empty and whitespace-only values, and a list of suspect markers; also look at the most frequent values.

Q Why does `col('x') == None` not find nulls?

A A null is never equal to anything, even another null, so the comparison is null, not True. Use `isNull()`.

Q dropna or fillna - how do you choose?

A dropna removes rows and loses data; fillna keeps every row with a default. Use `subset` so only the intended columns are touched, and decide per column whether a default is honest.

Q Where should marker handling happen?

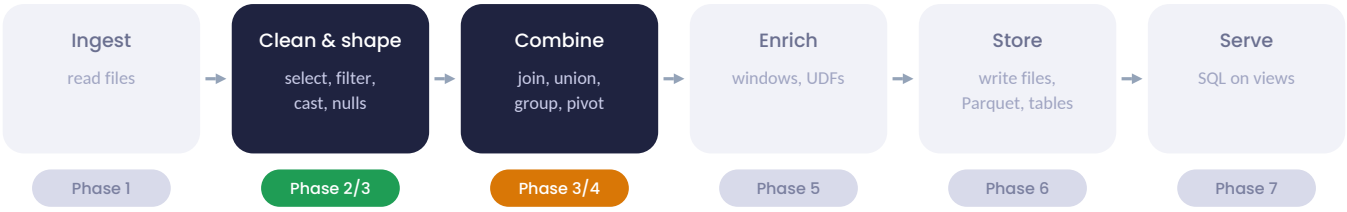
A As early as possible after landing the raw data (Bronze to Silver), in one tested function, with the marker list documented.

PHASE 3 CHECKPOINT

- I know union matches by position and unionByName matches by name, and I can show the bug that union causes.
- I can use upper / lower / initcap on text.
- I can add, subtract and compare dates and format them with date_format (and I know MM is months).
- I can drop or fill nulls, for all columns or a subset.
- I can split a string into an array, pick an element with an index, explode an array into rows and test it with array_contains.
- I can explain why `\N`, `NA` and the word null are not nulls to Spark.
- I can use `nullValue` at read time or a `when / isin` cleaner after it.
- I can profile real nulls against markers before deciding how to clean.

From notebook to pipeline

Data engineers use these same skills inside **pipelines**. Here is where this phase sits, and how it is used in practice.



DATA ENGINEERING LENS

unionByName is the safe way to append a new batch of data whose columns may be in a different order - plain union would silently shuffle values into the wrong columns.

Null handling is a **data-quality decision**: dropna removes rows (you lose data), fillna keeps every row with a default such as 'NotAvailable'. Use subset so you only touch the columns you mean.

The notebook adds **current_date()** as a new column - the same idea as stamping every row with a load date.

split + explode flatten array-like text so each element becomes a row that can be counted, filtered or joined.

Beyond the notebook: markers like **\N**, NA and the word null pass every isNull check. Convert them once at the Bronze to Silver boundary and document the marker list.

Glossary

Concepts from this phase, then every **code term** the decoder explained, in the order you met them.

Concept	In plain words
Union (by position)	Stacking rows; column 1 meets column 1, column 2 meets column 2.
Union by name	Stacking rows; columns are matched by their names.
Null	A missing value; never equal to anything, even another null.
Array column	A single cell that holds a list of values.
Explode	Turning every element of an array into its own row.
Missing-value marker	Text such as \N, NA or null that means unknown but is read as ordinary data.
nullValue	CSV option naming the text to read as a real null.

Code terms decoded in this phase

Code term	Meaning
`[('1','kad'), ...]`	A Python list of tuples. Each tuple is one row of the DataFrame.
`STRING / DOUBLE / INT`	SQL-style type names: text, decimal number, whole number.
`spark`	The SparkSession - your entry point to Spark. Databricks creates it for you, already named **spark**.
`spark.createDataFrame(rows, schema)`	Builds a DataFrame from Python data.
`spark.createDataFrame(data, schema)`	Builds a small DataFrame from a Python list plus a schema string.
`.union(other)`	Stacks the rows of two DataFrames, matching columns by **position**.
`.display()`	Databricks' table viewer. It is an **action**, so it makes Spark run the job.
`.unionByName(other)`	Stacks the rows of two DataFrames, matching columns by **name**.
`.select(...)`	Keeps only the listed columns (or new calculated columns) - like SQL SELECT.
`upper(col)`	Converts text to CAPITALS.
`.alias('new')`	Gives a column a new name **in this result only**.
`.withColumn(name, expression)`	**Adds** a column, or **replaces** the one with the same name.
`current_date()`	Today's date (a date, no time).
`date_add(col, n)`	Adds n days to a date. A **negative** n goes back.
`date_sub(col, n)`	Subtracts n days from a date.
`datediff(end, start)`	Days between two dates: **end minus start**.
`date_format(col, 'dd-MM-yyyy')`	Shows a date as **text** using a pattern. **MM** = month, **mm** = minutes.
`.dropna(how / subset)`	Drops rows containing nulls: **'any'** null, **'all'** nulls, or only in the *subset* columns.
`subset=[...]`	Restricts the operation to the listed columns only.
`.fillna(value, subset)`	Replaces nulls with a value (a text value only fills string columns).
`split(col, pattern)`	Cuts a string into pieces and returns an **array** column.
`...[1]`	Array index. Counting starts at **0**, so [1] is the second element.
`explode(col)`	Turns each element of an array into its **own row**.
`array_contains(col, value)`	True / False: does the array hold this value?
`spark.read`	Starts a **DataFrameReader**: the object you configure and then use to load a file.
`.format('csv')`	Chooses the file format: **csv**, **json**, **parquet** ... (the same call is used when writing).
`\ (backslash)`	Line continuation: the statement carries on in the next line. Nothing may follow it on the same line.
`.option(key, value)`	Sets **one** setting for the reader or writer. Call it again for each extra setting.
`header`	**True** = the first row of the CSV holds column names, not data.

Code term	Meaning
`nullValue`	CSV option: the text to read as a real null (for example \N).
`.load(path)`	Points Spark at the file or folder and returns a DataFrame. (Lazy until an action needs the data.)
`/FileStore/tables/`	The Databricks folder (DBFS) where uploaded files live.
`def name(args):`	Defines an ordinary Python function.
`when(condition, value)`	An if / else-if branch for a new column - like SQL CASE WHEN. First match wins.
`trim(col)`	Removes spaces from both ends of a string.
`col('name')`	Refers to a column by name so you can do something with it (rename, compare, calculate).
`.isin(a, b)`	True when the value is in the given list.
`.otherwise(value)`	The **else**: used for every row that matched no earlier when().
`sum(col)`	Total of a column (Spark's own *sum*, available through the import *).
`.isNull()`	True where the value is missing. (A null is never *equal* to anything, so == *None* fails.)
`.agg(...)`	Calculates one or more aggregates for every group.

Check yourself

Try each question first, then click **Show answer**. Every answer has a **back** link that returns you to the question.

Questions

Q1 When does `union` give wrong data without an error? Show answer

Q2 Do `date_sub('d', 7)` and `date_add('d', -7)` differ? Show answer

Q3 What type does `date_format` return, and what is the difference between `MM` and `mm`? Show answer

Q4 Compare `dropna('any')`, `dropna(subset=['Outlet_Size'])` and `fillna('NotAvailable')`. Show answer

Q5 What does explode do to the row count, and what is the risk? Show answer

Q6 Why does `col('w').isNull()` miss a value of `\N`? Show answer

Q7 Why can `cast('double')` on a marker fail in Spark 4? Show answer

Hands-on tasks

Task 1 Add a column due_date that is 14 days from today, shown as dd-MM-yyyy. Show solution

Task 2 Split Outlet_Type on spaces and add a True/False flag for rows that contain 'Type1'. Show solution

Task 3 Convert the markers \N and NA in Outlet_Size to nulls and count the nulls. Show solution

Answers and solutions

A1 When the two DataFrames have the same column types but a different column order: union matches by position, so values land in the wrong columns. Use `unionByName`. back to question

A2 No - both give the date 7 days earlier. back to question

A3 It returns a **string**. MM is the month, mm is minutes. back to question

A4 The first drops rows with any null; the second drops rows with a null in Outlet_Size only; fillna keeps every row and replaces nulls in **string** columns (use subset to limit it). back to question

A5 It makes one row per array element, so the count grows. Aggregating afterwards can count the same item more than once. back to question

A6 In the file it is just text, so Spark reads a string. Use `nullValue` or convert markers to null first. back to question

A7 ANSI mode is on by default, so a malformed value raises CAST_INVALID_INPUT. `try_cast` returns null instead. back to question

Task solutions

Solution 1 back to task

SOLUTION

```
df = df.withColumn('due_date', date_format(date_add(current_date(), 14), 'dd-MM-yyyy'))
```

Solution 2 back to task

SOLUTION

```
df_exp = df.withColumn('Outlet_Type', split('Outlet_Type', ' '))

df_exp.withColumn('Type1_flag', array_contains('Outlet_Type', 'Type1')).display()
```

Solution 3 back to task

SOLUTION

```
df_clean = markers_to_null(df, ['Outlet_Size'], ['\\N', 'NA'])
print(df_clean.filter(col('Outlet_Size').isNull()).count())
```

PREVIOUS
← Phase 2 · Beginner Transformations

NEXT
Phase 4 · Advanced Transformations →

Tip: keep all nine PDFs in the same folder so the previous / next links open the right file.